ASSIGNMENTNO. 2

PROCESS CONTROL SYSTEM CALLS

AIM : The demonstration of FORK and WAIT system calls along with zombie and orphan states.

Implement the C program in which main program accepts the integers to be sorted. Main program uses the FORK system call to create a new process called a child process. Parent process sorts the integers using sorting algorithm and waits for child process using WAIT system call to sort the integers using any sorting algorithm. Also demonstrate zombie and orphan states.

OBJECTIVES :

To study

1. Process control
2. Zombie and orphan processes
3. System calls : fork, execve, wait

THEORY :

Process

A process is the basic active entity in most operating-system models. A process is a program in execution in memory or in other words, an instance of a program in memory. Any program executed creates a process. A program can be a command, a shell script, or any binary executable or any application.

Process IDs

Each process in a Linux system is identified by its unique *process ID*, sometimes referred to as *pid*. Process IDs are 16-bit numbers that are assigned sequentially by Linux as new processes are created.

When referring to process IDs in a C or C++ program, always use the pid_t typedef, which is defined in <sys/types.h>. A program can obtain the process ID of the process it's running in with the getpid() system call, and it can obtain the process ID of its parent process with the getppid() system call.

Creating Process

Two common techniques are used for creating a new process.

1. using system() function
2. using fork() system calls

1. system() function

The system function in the standard C library provides an easy way to execute a command from within a program, much as if the command had been typed into a shell. In fact, system creates a subprocess running the standard Bourne shell (/bin/sh) and hands the command to that shell for execution.

The system function returns the exit status of the shell command. If the shell itself cannot be run, system returns 127; if another error occurs, system returns –1.

2.fork system call

A process can create a new process by calling fork. The calling process becomes the parent, and the created process is called the child. The fork function copies the parent's memory image so that the new process receives a copy of the address space of the parent. Both processes continue at the instruction after the fork statement (executing in their respective memory images).

Synopsis

```
#include <unistd.h>
  pid_t  fork(void);
```

The fork function returns 0 to the child and returns the child's process ID to the parent. When fork fails, it returns –1.

Wait Function

When a process creates a child, both parent and child proceed with execution from the point of the fork. The parent can execute wait to block until the child finishes. The wait function causes the caller to suspend execution until a child's status becomes available or until the caller receives a signal.

Synopsis

```
#include <sys/wait.h>
  pid_t wait(int *status);
```

If wait returns because the status of a child is reported, these functions return the process ID of child. If an error occurs, these functions return –1.

Status values

The status argument of wait is a pointer to an integer variable. If it is not NULL, this function stores the return status of the child in this location. The child returns its status by calling exit, _exit or return from main.

Exec system call

Used for new program execution within the existing process. The fork function creates a copy of the calling process, but many applications require the child process to execute code that is different from that of the parent. The exec family of functions provides a facility for overlaying the process image of the calling process with a new image. The traditional way to use the fork–exec combination is for the child to execute (with an exec function) the new program while the parent continues to execute the original code. The exec system call is used after a fork system call by one of the two processes to replace the memory space with a new program. The exec system call loads a binary file into memory (destroying image of the program containing the exec system call) and go their separate ways.

All exec functions return –1 if unsuccessful. In case of success these functions never return to the calling function.

Process Termination

Normally, a process terminates in one of two ways. Either the executing program calls the exit function, or the program's main function returns. Each process has an exit code: a number that the process returns to its parent. The exit code is the argument passed to the exit function, or the value returned from main.

Zombie Processes

If a child process terminates while its parent is calling a wait function, the child process vanishes and its termination status is passed to its parent via the wait call. But what happens, when a child process terminates and the parent is not calling wait? Does it simply vanish? No, because then information about its termination—such as whether it exited normally and, if so, what its exit status is—would be lost. Instead, when a child process terminates, it becomes a zombie process. A *zombie process* is a process that has terminated but has not been cleaned up yet. It is the responsibility of the parent process to clean up its zombie children. The wait functions do this, too, so it's not necessary to track whether your child process is still executing before waiting for

it. Suppose, for instance, that a program forks a child process, performs some other computations, and then calls wait. If the child process has not terminated at that point, the parent process will block in the wait call until the child process finishes. If the child process finishes before the parent process calls wait, the child process becomes a zombie. When the parent process calls wait, the zombie child's termination status is extracted, the child process is deleted, and the wait call returns immediately.

Orphan Process

Orphan process is the process whose parent process is dead. That is, parent process is terminated, killed or exited but the child process is still alive.

A process can be orphaned either intentionally or unintentionally. Sometimes a parent process exits/terminates or crashes leaving the child process still running, and then they become orphans. Also, a process can be intentionally orphaned just to keep it running. For example when you need to run a job in the background which need any manual intervention and going to take long time, then you detach it from user session and leave it there. Same way, when you need to run a process in the background for infinite time, you need to do the same thing. Processes running in the background like this are known as daemon process.

Comparison with fork

1. Creates new process without fully copying the address space of the parent.
2. fork guarantees that the child runs first, until the child calls exec or exit.
3. When child calls either of these two functions (exit, exec), the parent resumes.

Input

1. An integer array with specified size.
2. An integer array with specified size and number to search.

Output

1. Sorted array.
2. Status of number to be searched.

FAQs

- Is Orphan process different from a Zombie process?
- Are Orphan processes harmful for system?
- Is it bad to have Zombie processes on your system?
- How to find an Orphan Process?
- How to find a Zombie Process?
- What is common shared data between parent and child process?
- What are the contents of Process Control Block?

Practice Assignments

Example 1

Printing the Process ID

```
#include <stdio.h>
#include <unistd.h>
        int main()
        {
        printf("The process ID is %d\n", (int) getpid());
        printf("The parent process ID is %d\n", (int) getppid());
        return 0;
        }
```

Example 2: **Using fork to duplicate a program's process**

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
        int main()
        {
                pid_t child_pid;
                printf("The main program process ID is %d\n", (int) getpid());
                child_pid=fork();
        if (child_pid !=0)
        {
                Printf ("This is the parent process ID, with id %d\n", (int)
                getpid());
                Printf ("The child process ID is %d\n", (int) child_pid);
        }
        else
                printf("This is the child process ID, with id %d\n", (int) getpid());
                return 0;
        }
```

Example 3

Making a zombie process

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

int main()
{
   pid_t pid;

   pid=fork();

   if(pid==0)
   {
      printf("Child Process Executed.\n");
      exit(0);
   }
   else
   {
```

Example 4: Demonstration of orphan process

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main()
{   pid_t pid;
   pid=fork();
   if(pid==0)
   {  sleep(8);
      printf("\nChild Process\n");
      printf("Child PID  : %d\n",getpid());
      printf("Parent PID : %d\n",getppid());
   }   else{
      printf("Parent Process Exiting...\n");
      exit(0);
   }
   return 0;
}}
```

Program 2:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>

void bubbleSort(int a[], int n)
{
    int i,j,temp;

    for(i=0;i<n-1;i++)
    {
        for(j=0;j<n-i-1;j++)
        {
            if(a[j]>a[j+1])
            {
                temp=a[j];
                a[j]=a[j+1];
                a[j+1]=temp;
            }
        }
    }
}

void selectionSort(int a[], int n)
```

```
{
    int i,j,min,temp;

    for(i=0;i<n-1;i++)
    {
        min=i;

        for(j=i+1;j<n;j++)
        {
            if(a[j]<a[min])
                min=j;
        }

        temp=a[min];
        a[min]=a[i];
        a[i]=temp;
    }
}

void display(int a[],int n)
{
    int i;

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    printf("\n");
}

int main()
{
    int a[100],n,i;
    pid_t pid;

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter elements:\n");

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    pid=fork();

    if(pid<0)
    {
        printf("Fork Failed\n");
        exit(1);
    }

    else if(pid==0)
    {
        printf("\n----- Child Process -----\n");
        printf("Child PID : %d\n",getpid());

        selectionSort(a,n);

        printf("Sorted Array using Selection Sort:\n");
        display(a,n);
```

```
        exit(0);
    }

    else
    {
        wait(NULL);

        printf("\n----- Parent Process -----\n");
        printf("Parent PID : %d\n",getpid());

        bubbleSort(a,n);

        printf("Sorted Array using Bubble Sort:\n");
        display(a,n);
    }

    return 0;
}
```

```
      for(i=0;i<size;i++)
        {
             printf("Enter number %d:",(i+1));
             scanf("%d",&a[i]);
             pArr[i]=a[i];
             cArr[i]=a[i];

         }


         printf("Your Entered Integers for Sorting\n");
         display(a,size);


             pid=getpid();
             printf("Current Process ID is : %d\n",pid);

            printf("[ Forking Child Process ... ] \n");
             child_pid=fork();

         if( child_pid < 0)
          {

              printf("\nChild Process Creation Failed!!!!\n");
                     exit(-1);
             }
             else if( child_pid==0) {

             printf("\nThe Child Process\n");
             printf("\nchild process is %d",getpid());
             printf("\nparent of child process is %d",getppid());
             printf("Child is sorting the list of Integers by QUICK
             SORT::\n");
             quickSort(cArr,0,size-1);
             printf("The sorted List by Child::\n");
             display(cArr,size);
             printf("Child Process Completed ...\n");
             sleep(10);
                  printf("\nparent of child process is %d",getppid());
             }

             else {

             printf("parent process %d started\n",getpid());
                 printf("Parent of parent is %d\n",getppid());

             sleep(30);
             printf("The Parent Process\n");
             printf("Parent %d is sorting the list of Integers by MERGE
SORT\n",pid);
             partition(pArr,0,size-1);
             printf("The sorted List by Parent::\n");
             display(pArr,size);

                  printf("Parent Process Completed ...\n");
             }

    return 0;
}
```

```
int split ( int a[ ], int lower, int upper )
{
    int i, p, q, t ;

    p = lower + 1;
    q = upper ;
    i = a[lower] ;

    while ( q >= p )
    {
        while ( a[p] < i )
            p++ ;

        while ( a[q] > i )
            q-- ;

        if ( q > p )
        {
            t = a[p] ;
            a[p] = a[q] ;
            a[q] = t ;
        }
    }

    t = a[lower] ;
    a[lower] = a[q] ;
    a[q] = t ;

    return q ;
}

void quickSort(int a[],int lower, int upper){
     int i ;
        if ( upper > lower )
        {
            i = split ( a, lower, upper ) ;
            quickSort ( a, lower, i - 1 ) ;
            quickSort ( a, i + 1, upper ) ;
        }
}
```